

Relatório

1. Alunos

- Daniel Wesley Freitas Siqueira GRR20245621
- Ulisses Bastian Machado da Rosa GRR20245567

2. Link Para o github

3. Uso

```
make
# o arquivo csv é opcional
# caso não seja providenciado um csv, o programa procura
por ufrpr.csv automaticamente
sudo ./blind_pacman --server <interface> [<arquivo>.csv]
sudo ./blind_pacman --client <interface>
```

4. Implementação

4.1. Kermit

Foi criada a seguinte estrutura de dados para representar um pacote do protocolo *Kermit*:

```
struct KermitPacket {
    struct {
        unsigned char init_marker = KERMIT_INIT_MARKER;
        unsigned char size : 5;
        unsigned char sequence : 6;
        PacketType type : 5; // enum com os tipos de mensagem
    } header;
    char data[BUFFER_SIZE + 1] // esse vetor tem tamanho
    suficiente para guardar uma mensagem de tamanho até 32B +
    1B
}
```

4.1.1. Construção do pacote

- Dados são escritos na seção **data**
- O CRC é calculado sobre o **header** inteiro e sobre **data** (até **32B**) e é escrito após o último byte relacionado ao pacote, ou seja, se o tamanho indicado for **5**, por exemplo, o **CRC** será escrito no sexto byte do vetor.
- O conteúdo binário do struct é copiado para um buffer maior, com os bytes **x81** e **x88** sendo escapados com **xff**.
- Pacotes enviados têm tamanho mínimo de **sizeof(header) + 1B** e tamanho máximo **sizeof(header) + 31B + 1B**.

Para envio de pacotes foram feitas duas funções:

```
// função que recebe um pacote e guarda na estrutura
KermitPacket;
// utiliza recv() da libc e, em caso de timeout, retorna
erro (que é utilizado mais à frente);
// também retorna outros erros que podem ser relevantes
(sequencia errada, crc errado, etc.)
PacketError KermitPacket::receivePacket(int socket);
// função que envia os dados presentes na estrutura;
// utiliza send() da libc, e retorna erro caso haja erro
com essa função
PacketError KermitPacket::sendPacket(int socket);
```

4.2. Envio de mensagens

Para facilitar o processo de envio de mensagens, foi criada uma camada de abstração com 3 funções:

```
// recebe pacotes de uma mensagem e concatena o conteúdo em um
std::vector
PacketError KermitPacket::receive(int socket, std::vector<char*>
buffer);
// "quebra" os dados presentes em um buffer de bytes e envia pacotes
referentes a esses dados
// inicia a transmissão com uma mensagem de tipo específico
initialize
PacketError KermitPacket::send(int socket, PacketType type, const
char* data, unsigned int data_size);
// chamada sempre após send() para poder "trocar" o estado do
emissor e receptor
// responsável por checar se o outro lado recebeu o pacote do tipo
end_transmission
PacketError KermitPacket::confirmSend(int socket);
```

4.2.1. Timeout

A função **send()** também implementa um segundo timeout, pois podemos receber “lixo” dos raw sockets com **recv()**. Dessa forma, se ficarmos durante **SEND_TIMEOUT_SECS** segundos (atualmente 3s) sem receber mensagens do protocolo *Kermit*, podemos considerar que o envio falhou e reenviamos o pacote atual.

4.2.2. Sincronização

Com intuito de sincronizar ambos os lados da transmissão, a função **confirmSend()** foi feita para “trocar” o estado do emissor e receptor, ou seja, é assumido que quem recebe com **receive()** passará a poder enviar após o emissor chamar **confirmSend()** e quem envia passará a ouvir depois que chamar **confirmSend()**.

Nos casos em que um dos lados precisa mandar duas mensagens seguidas (ex.: servidor quer enviar um arquivo → nome + dados), o lado receptor identifica a mensagem e envia um “dummy” de volta para retomar os estados anteriores e receber a próxima mensagem. Essas mensagens “compostas” são iniciadas por um tipo específico (arquivo ou visualização) e seguidas por mensagens com pacotes do tipo **data** (além do início e final).

4.2.3. Mensagens

- Mensagens podem ter tamanho variável.
- Cada mensagem é composta por pacotes:
 - ▶ Pacote inicial.
 - ▶ Número variável de pacotes intermediários.
 - ▶ Pacote final. (enviada por **confirmSend()**)
- Cada pacote segue o protocolo *Kermit*.

4.3. Pacman

- A visualização do PacMan é um quadrado em volta dele, ou seja, se o nível da visualização é 2, então o quadrado resultante terá dimensões 5 × 5 com o PacMan em seu centro.
- A IA dos fantasmas foi feita utilizando
 - ▶ Mão direita
 - ▶ Mão esquerda
 - ▶ Alternante (mão esquerda/direita)
 - ▶ Aleatório
- As posições das entidades dependem do que há no csv:
 - ▶ Se qualquer uma delas for definida já no csv, a posição se mantém.
 - ▶ As que não estiverem no csv são aleatoriamente determinadas após a leitura do grid.

4.4. Cliente e Servidor

- Servidor começa enviando
- Cliente começa escutando
- A comunicação é considerada como estabelecida quando o cliente recebe de fato a primeira mensagem (por meio de **receive()**).
- Grid:
 - ▶ O servidor envia uma mensagem do tipo visualização, que contem as dimensões do grid
 - ▶ O cliente guarda a informação das dimensões e envia um “dummy” para voltar a ouvir
 - ▶ O servidor envia o grid com a visão do pacman
- Arquivos:
 - ▶ O servidor envia uma mensagem que especifica o tipo do arquivo e que contém seu nome.
 - ▶ O cliente, após receber uma mensagem com o nome do arquivo (tipo), cria um arquivo temporário **nome ++ _received.<tipo>**.
 - ▶ Após a criação do arquivo, o cliente envia um “dummy” para receber o conteúdo e escrevê-lo no arquivo que criou.
 - ▶ Os arquivos recebidos pelo cliente são apagados assim que a visualização atual do arquivo é fechada.